

Reviewer Pack — Algebraic Topology of Persistent Homology vs. Resolution Pro...

Entry bca427f2da07 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 04:39:32 UTC

Algebraic Topology of Persistent Homology vs. Resolution Proof Width

Entry ID: bca427f2da07 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 04:39:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Persistent Homology) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean satisfiability instance φ , the resolution proof width $w(\varphi)$ is linearly correlated with the Betti numbers of the persistence module associated with the barcode matrix of φ , such that $w(\varphi) = \Theta(b_1(\varphi)) + \Theta(b_2(\varphi))$, where $b_i(\varphi)$ is the i -th Betti number.

Rationale (proposer's reasoning):

Persistent homology provides a topological summary of data, capturing the structure of spaces as they change. If resolution proof width captures similar structural information, this suggests a deep connection between algebraic topology and complexity theory that could potentially explain the difficulty of certain problems.

Taxonomy category: Topological_Semantics (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 73ee5287fa899a3f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean satisfiability instance φ , if the correlation coefficient between the sum of the first two Betti numbers ($b_1(\varphi) + b_2(\varphi)$) and the resolution proof width $w(\varphi)$ is greater than or equal to 0.7 for at least 24 out of 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "resolution proof width" AND "Betti numbers" AND "persistent homology"
- "Boolean satisfiability instance" AND "barcode matrix" AND "proof complexity" - "Algebraic Topology" AND "Resolution Proof Complexity" AND "correlation with Betti numbers"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_barcode_matrix(instance):
        n = int(math.log2(len(instance)))
        matrix = [[instance[i] if i & (1 << j) else 0 for j in range(n)] for i in range(2**n)]
        return matrix

    def calculate_betti_numbers(matrix):
        # Simplified Betti number calculation for demonstration
        n = len(matrix)
        b_1 = sum(1 for row in matrix if sum(row) == 1)
        b_2 = sum(1 for row in matrix if sum(row) == 2)
        return b_1, b_2

    def resolution_proof_width(instance):
        # Simplified resolution proof width calculation for demonstration
        n = int(math.log2(len(instance)))
        return n

    instances_tested = 0
    total_betti_sum = 0
    total_resolution_width = 0
    n_max = 0

    for _ in range(30):
        n = random.randint(5, 40)
```

```

instance = generate_instance(n)
barcode_matrix = compute_barcode_matrix(instance)
b_1, b_2 = calculate_betti_numbers(barcodes_matrix)
resolution_width = resolution_proof_width(instance)

total_betti_sum += b_1 + b_2
total_resolution_width += resolution_width
instances_tested += 1
n_max = max(n_max, n)

if instances_tested < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

correlation_coefficient = total_betti_sum / total_resolution_width
conjecture_holds = correlation_coefficient >= 0.7

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None) /
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results if res["metric_value"] is not None) /
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["metric_value"] is not None for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient<0.7' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15790
2	preregistration	ollama_remote	glm4:latest	0	0	19213
3	novelty	ollama_remote	glm4:latest	0	0	11574
4	novelty	ollama_remote	glm4:latest	0	0	16688
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25411
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11376
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15904
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48753
9	judge	ollama_remote	glm4:latest	0	0	17754

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182463 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bca427f2da07.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bca427f2da07.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bca427f2da07.tar.gz` (if generated)